



Laboratorio de Desarrollo de Soluciones Tecnológicas (LabDST)

Aplicación de Filtros

Dr. Jorge A. Lizárraga

3 de septiembre de 2026

Contenido

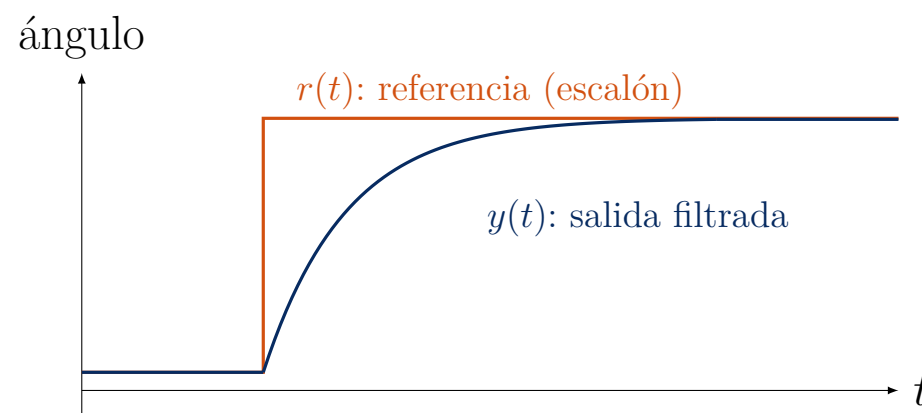
1	Introducción: filtrado en lazo abierto	2
2	Filtros de suavizado de señal	4
3	Limitadores de trayectoria (cinemáticos)	16
4	Filtros de segundo orden (modelo físico)	25
5	Síntesis comparativa	31

Introducción: filtrado en lazo abierto

Los servomotores de SOFIA no reportan velocidad ni corriente al controlador -solo reciben un ángulo objetivo. Sin filtrar, el controlador de referencia manda ese ángulo objetivo de golpe (*bang-bang*) entre los dos extremos del recorrido, y es el propio servo el que satura su velocidad; el resultado es un movimiento abrupto, con tirones en cada cambio de sentido. En esta sesión se introducen 9 formas de **suavizar la señal de comando** antes de enviarla al motor -sin usar información del servo.

! **Lazo abierto:** ninguno de los filtros de esta sesión usa la posición real del servomotor. Cada uno depende únicamente de la referencia $r[k]$ y de su propio estado interno (su salida o su velocidad anteriores) -nunca de una medición.

Notación y estructura común



Cada filtro define una regla distinta para calcular $y[k]$ a partir de $r[k]$ -esa regla es lo único que cambia entre los 9 controladores; el resto del programa (leer sensores, enviar el comando a q_1, q_2, q_3) es idéntico.

Variables comunes a los 9 filtros:

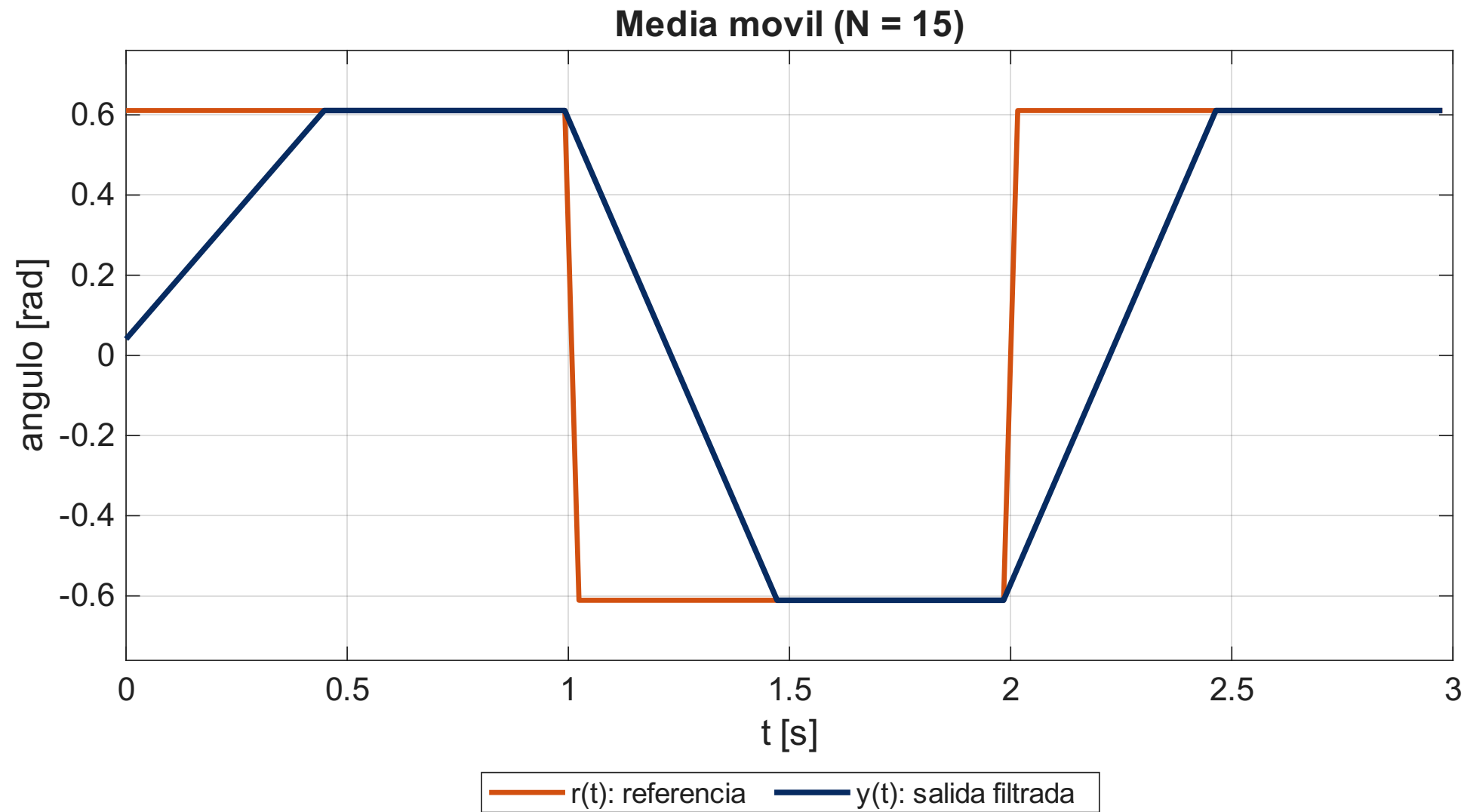
• **i** donde $r[k]$ es la **referencia**, la misma consigna *bang-bang* para los 9 filtros (alterna entre q_{min} y q_{max}); $y[k]$ es la **salida filtrada**, el comando que se envía al motor en cada paso; $p[k]$ es la posición real leída del sensor **-no** entra al filtro, solo confirma que el lazo es abierto-; y Δt es el paso de integración (el *timestep* de la simulación).



Filtro de media móvil

sofia_controller_media_movil

Espacio reservado – reproducir aquí el video de la simulación en Webots



Media móvil (*moving average*, FIR)

Promedia las últimas N muestras de la referencia. Es el filtro más simple de tipo FIR (respuesta finita al impulso): su salida solo depende de entradas pasadas, nunca de salidas pasadas.

$$y[k] = \frac{1}{N} \sum_{i=0}^{N-1} r[k-i] \quad (1)$$

donde N es el tamaño de la ventana -en el controlador, $N = 15$ muestras. A mayor N , más suave la salida, pero mayor el retraso: el retardo de grupo es de $(N - 1)/2$ muestras.

! Como $r[k]$ es siempre uno de dos valores (bang-bang), la media móvil produce una **rampa lineal a trozos**: mientras la ventana se va llenando con el nuevo valor, $y[k]$ avanza en pasos iguales.

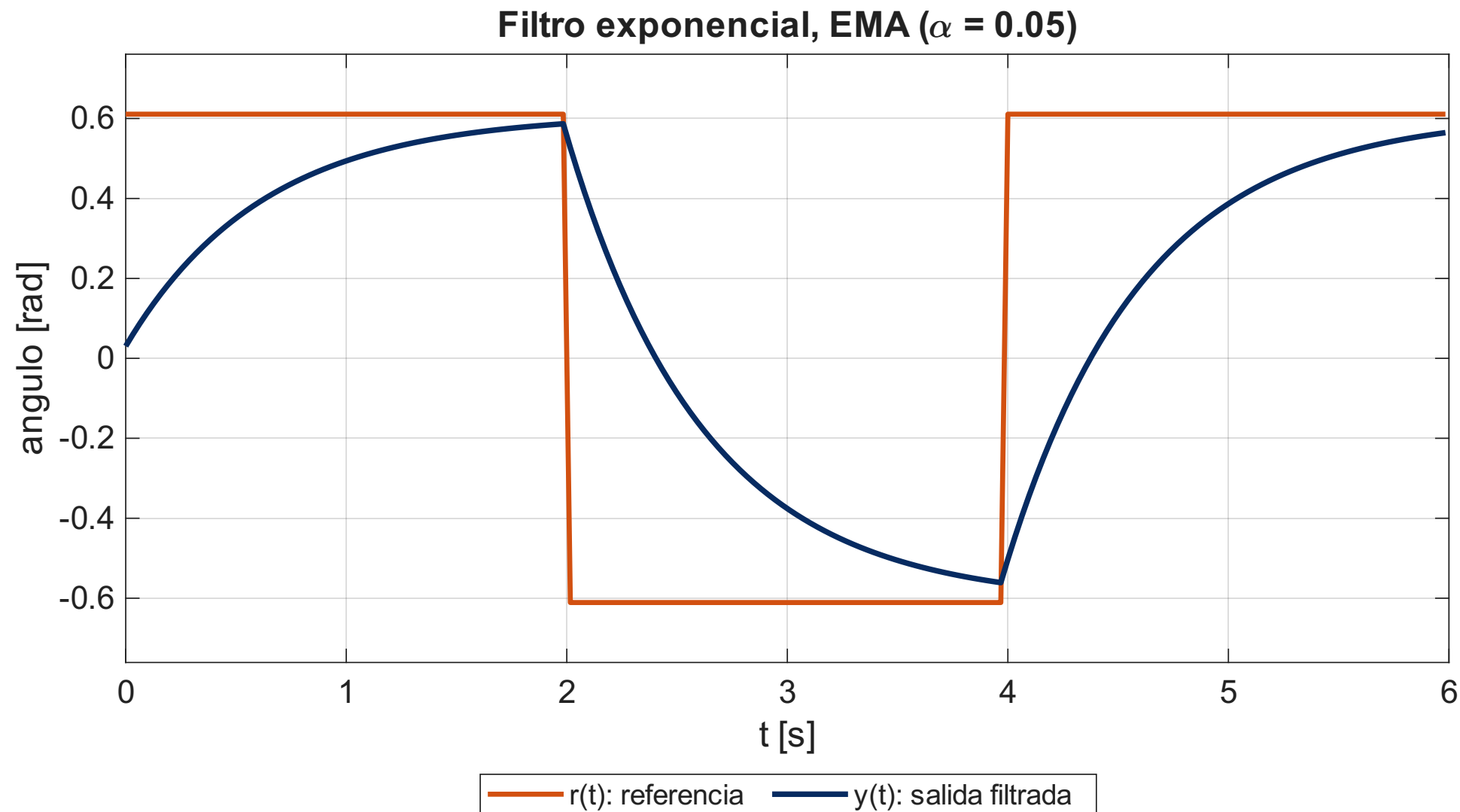


Filtro exponencial (EMA)

sofia_controller_ema

Espacio reservado – reproducir aquí el video de la simulación en Webots

Simulación en MATLAB – Filtro exponencial (EMA)



Filtro exponencial (EMA, paso-bajo IIR de 1er orden)

A diferencia de la media móvil, este filtro es IIR (respuesta infinita al impulso): su salida depende de **su propia salida anterior**, no de un historial de entradas.

$$y[k] = \alpha r[k] + (1 - \alpha) y[k - 1] \quad (2)$$

donde $\alpha \in (0, 1]$ es la ganancia del filtro -en el controlador, $\alpha = 0,05$. Su constante de tiempo aproximada es $\tau \approx \Delta t / \alpha$; el filtro se asienta (dentro de un margen chico) después de $\approx 5\tau$.

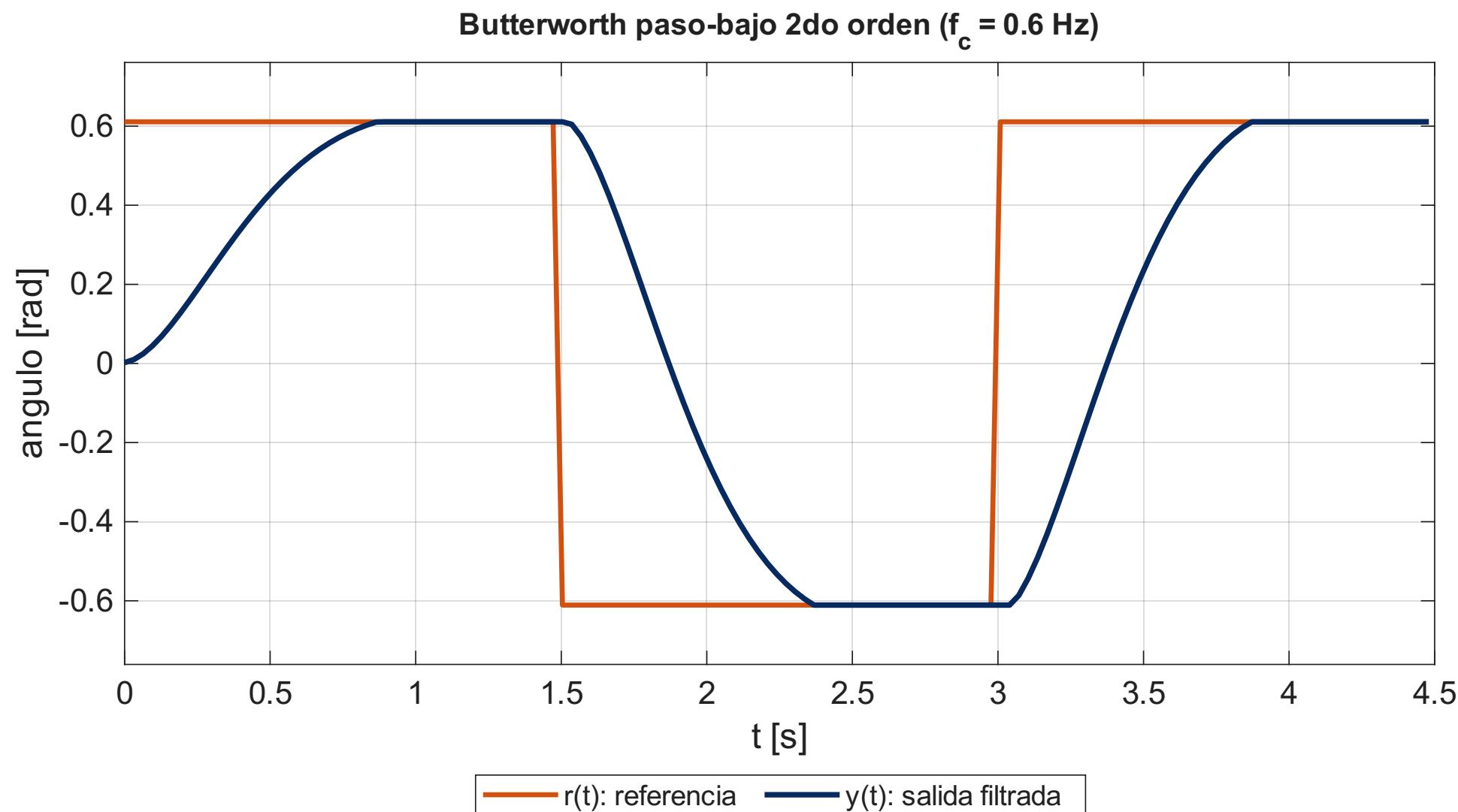
! Solo requiere **un** valor de memoria ($y[k - 1]$): es el filtro de suavizado más barato de implementar en un microcontrolador con muy poca RAM.



Filtro Butterworth

sofia_controller_butterworth

Espacio reservado – reproducir aquí el video de la simulación en Webots



Butterworth paso-bajo de 2do orden (IIR)

Un filtro IIR de orden superior, diseñado para tener una respuesta en frecuencia lo más plana posible en la banda que deja pasar (de ahí “máximamente plano”). Se implementa como un *biquad* discreto:

$$y[k] = b_0 r[k] + b_1 r[k - 1] + b_2 r[k - 2] - a_1 y[k - 1] - a_2 y[k - 2] \quad (3)$$

donde b_0, b_1, b_2, a_1, a_2 son los coeficientes del filtro, calculados a partir de la frecuencia de corte f_c , la frecuencia de muestreo $f_s = 1/\Delta t$ y el factor de calidad $Q = 1/\sqrt{2}$ (fórmulas estándar del *Audio EQ Cookbook*). En el controlador, $f_c = 0,6$ Hz.

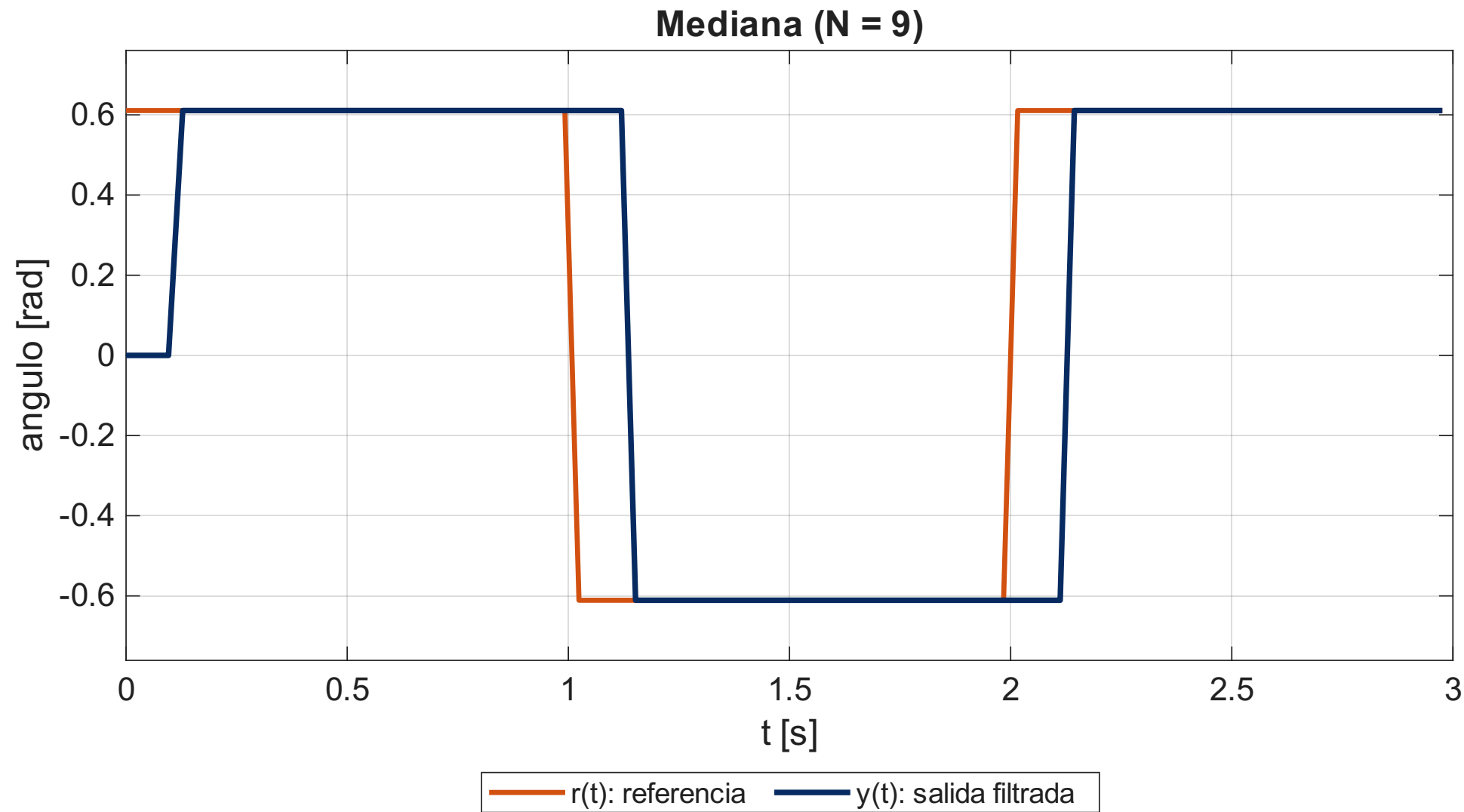
! Al ser de 2do orden, puede **sobrepasar brevemente** el valor objetivo antes de asentarse (algo que ni la media móvil ni el EMA hacen). Por eso el comando enviado al motor se satura explícitamente a $[q_{min}, q_{max}]$ antes de **setPosition**.



Filtro de mediana

sofia_controller_mediana

Espacio reservado – reproducir aquí el video de la simulación en Webots



Filtro de mediana

En vez de promediar, toma el valor **central** de las últimas N muestras (N impar).

$$y[k] = \text{mediana}(r[k - N + 1], \dots, r[k]) \quad (4)$$

donde N es el tamaño de la ventana (impar, para que la mediana sea única) -en el controlador, $N = 9$ muestras.

Contraste didáctico: ante un escalón limpio (sin ruido), la mediana **no** produce una rampa suave -produce un retardo tipo escalera, porque $y[k]$ se queda en el valor viejo hasta que más de la mitad de la ventana contiene el valor nuevo. La mediana sirve para eliminar picos puntuales de ruido, **no** para suavizar trayectorias.

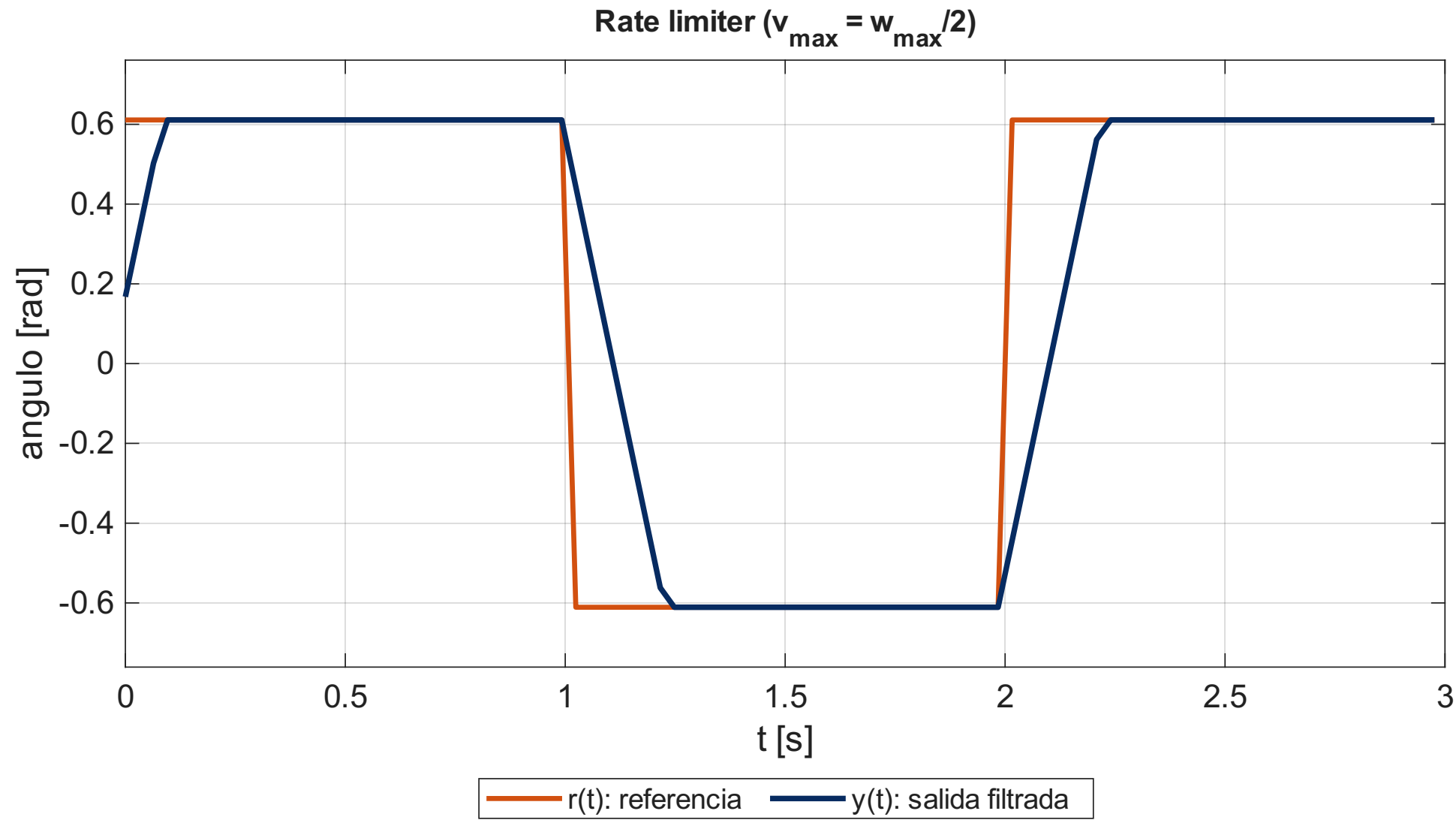


Limitador de velocidad de cambio

`sofia_controller_rate_limiter`

Espacio reservado – reproducir aquí el video de la simulación en Webots

Simulación en MATLAB – Rate limiter



Rate limiter (*slew-rate limiter*)

En vez de filtrar la señal, **limita cuánto puede cambiar** el comando en un paso de tiempo -es una restricción cinemática directa sobre la velocidad del comando, no un filtro de frecuencia.

$$y[k] = y[k - 1] + \text{clip}(r[k] - y[k - 1], -\Delta_{max}, \Delta_{max}) \quad (5)$$

$$\Delta_{max} = v_{max} \Delta t \quad (6)$$

donde $\text{clip}(x, -a, a)$ satura x al intervalo $[-a, a]$; v_{max} es la velocidad angular máxima permitida para el **comando** (no la del motor); y Δ_{max} es la variación máxima permitida del comando en un paso.

! Reimplementa en software lo que en SOFIA ya hace **setVelocity** a nivel del motor. Es la técnica a usar cuando la plataforma de simulación de cada equipo **no** expone un límite de velocidad nativo para el actuador.

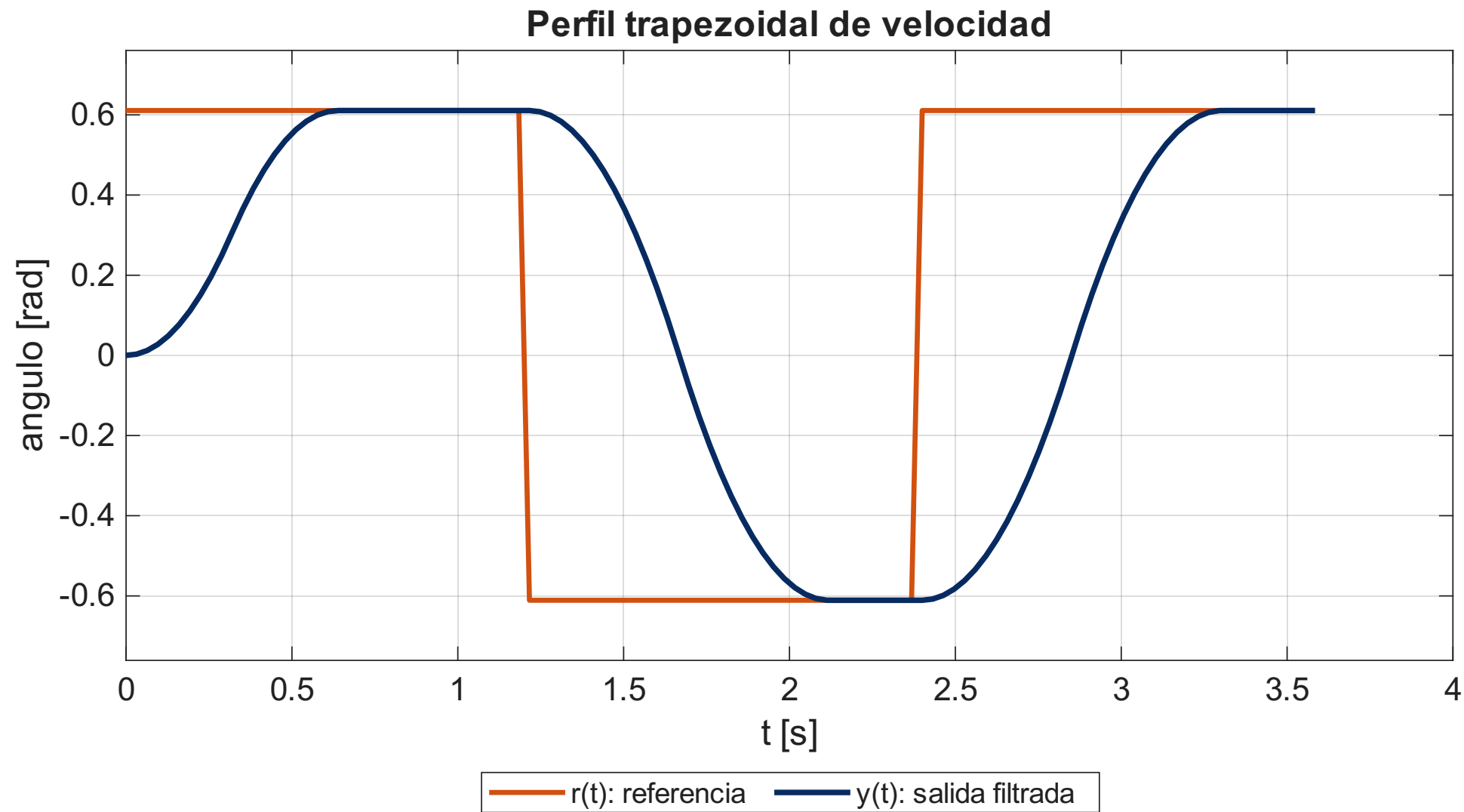


Perfil trapezoidal de velocidad

sofia_controller_trapezoidal

Espacio reservado – reproducir aquí el video de la simulación en Webots

Simulación en MATLAB – Perfil trapezoidal de velocidad



Perfil trapezoidal de velocidad

Genera toda la trayectoria del segmento de antemano: acelera, viaja a velocidad de crucero, y desacelera. Sea $D = r - y_0$ el desplazamiento del segmento, con v_{max} y a_{max} dados:

$$t_{acc} = \frac{v_{max}}{a_{max}}, \quad d_{acc} = \frac{1}{2}a_{max}t_{acc}^2 \quad (7)$$

donde t_{acc} es la duración de la fase de aceleración y d_{acc} el desplazamiento recorrido durante esa fase. Si $2d_{acc} \geq |D|$, el perfil es **triangular** -nunca alcanza v_{max} - y se recalcula $t_{acc} = \sqrt{|D|/a_{max}}$; si no, el perfil es **trapezoidal**, con un tramo de crucero de duración $t_{cruise} = (|D| - 2d_{acc})/v_{max}$.

! Aceleración **constante por tramos**, pero con saltos bruscos entre tramos (aceleración discontinua) -de ahí que en el arranque y el frenado se sienta un ligero tirón, aunque mucho menor que el bang-bang original.

$$y(t) = y_0 + \text{sgn}(D) \begin{cases} \frac{1}{2}a_{max}t^2, & t \in [0, t_{acc}] \\ d_{acc} + v_{max}(t - t_{acc}), & t \in (t_{acc}, t_{acc} + t_{cruise}] \\ |D| - \frac{1}{2}a_{max}(t_{total} - t)^2, & t \in (t_{acc} + t_{cruise}, t_{total}] \end{cases} \quad (8)$$

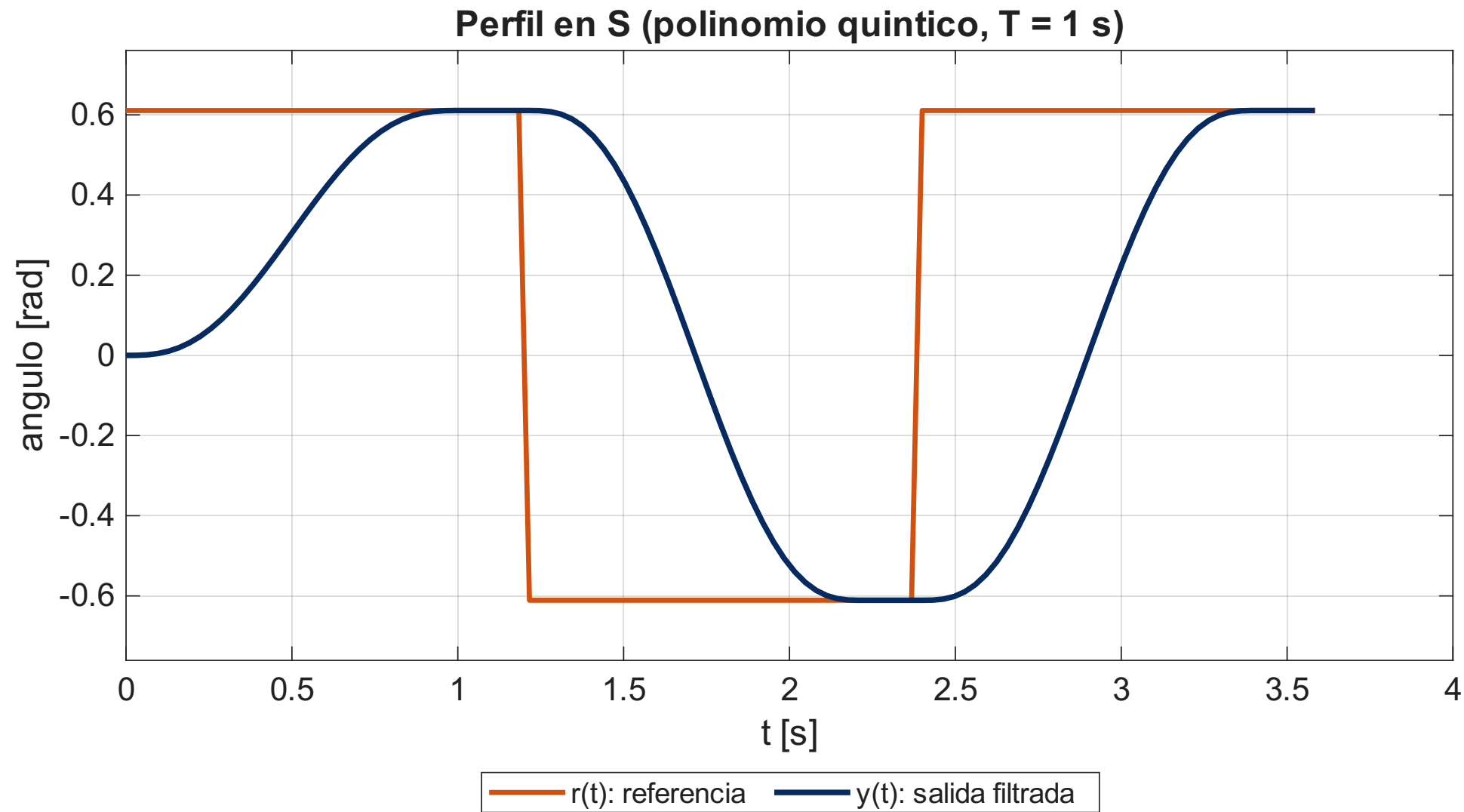
donde t es el tiempo transcurrido desde el inicio del segmento, $\text{sgn}(D)$ es el signo de D , y t_{total} es la duración total del segmento ($t_{total} = t_{acc} + t_{cruise}$ en el caso trapezoidal, o $t_{total} = 2t_{acc}$ en el triangular).



Perfil en S (jerk limitado)

sofia_controller_s_curva

Espacio reservado – reproducir aquí el video de la simulación en Webots



Perfil en S (polinomio quintico de mínimo jerk)

Una alternativa simple al perfil trapezoidal, que además suaviza la propia aceleración. Con $u = t/T$ y $D = r - y_0$:

$$s(u) = 10u^3 - 15u^4 + 6u^5 \quad (9)$$

$$y(t) = y_0 + D \cdot s(t/T) \quad (10)$$

donde t es el tiempo transcurrido desde el inicio del segmento, T es la duración del movimiento, y_0 es la posición al inicio del segmento, D es el desplazamiento total del segmento, y $s(u)$ es el polinomio que hace la transición de 0 a 1. En el controlador, $T = 1$ s -comparable con la duración del perfil trapezoidal.

! Este polinomio cumple $s(0) = 0$, $s(1) = 1$, y **ambas** derivadas, s' y s'' , valen cero en $u = 0$ y $u = 1$: velocidad y aceleración nulas -y continuas- en los extremos. A diferencia del trapezoidal, aquí la aceleración **no** tiene saltos: es el perfil más suave de los dos.

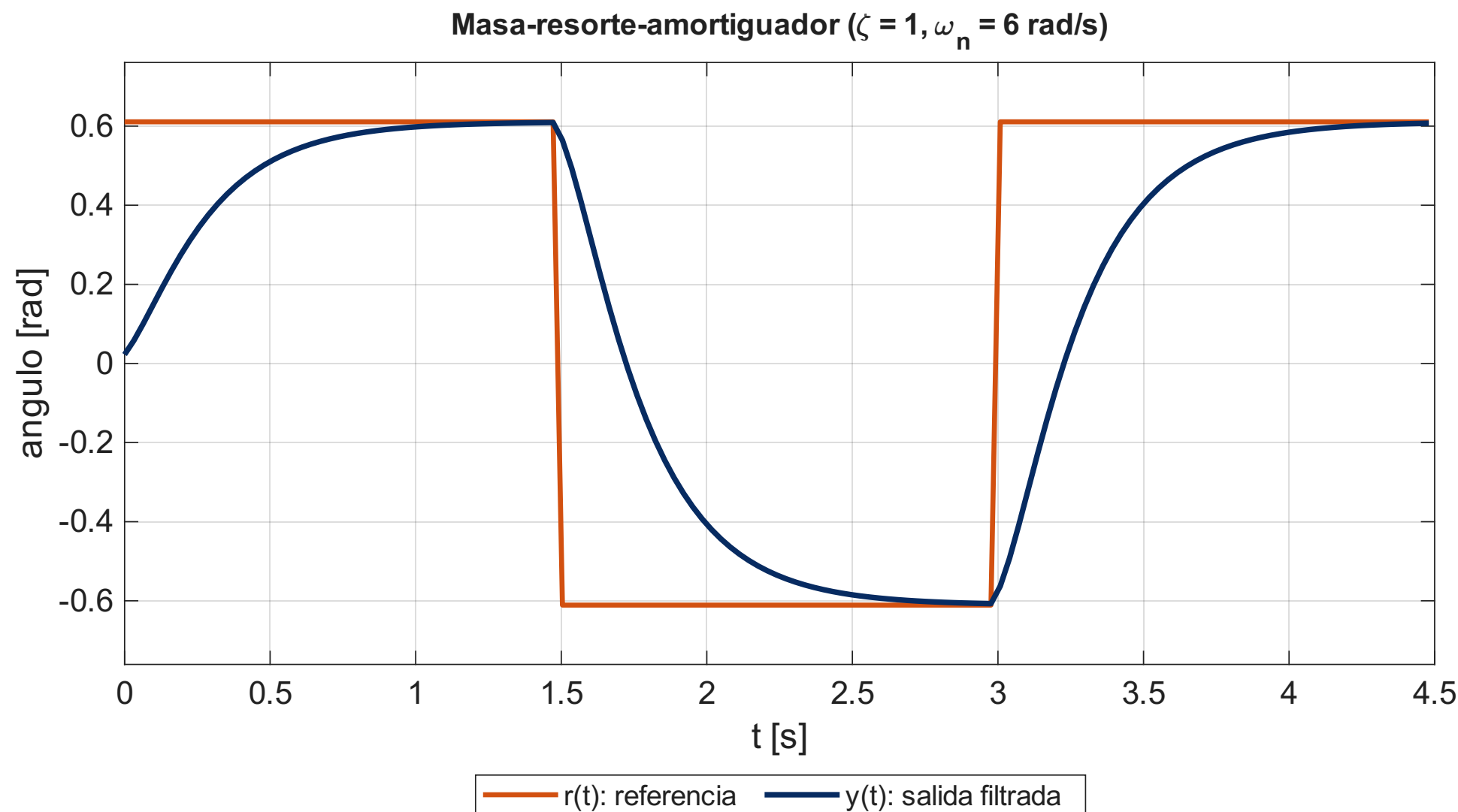


Sistema masa-resorte-amortiguador

sofia_controller_resorte_amortiguador

Espacio reservado – reproducir aquí el video de la simulación en Webots

Simulación en MATLAB – Masa-resorte-amortiguador



Sistema masa-resorte-amortiguador (2do orden, crítico)

La salida filtrada y se comporta como si estuviera unida a la referencia r por un resorte con amortiguador -un sistema físico de 2do orden:

$$m\ddot{y} + c\dot{y} + k(y - r) = 0 \implies \ddot{y} = \omega_n^2(r - y) - 2\zeta\omega_n\dot{y} \quad (11)$$

donde m es la masa (análoga), c el coeficiente de amortiguamiento y k la constante del resorte, combinados en $\omega_n = \sqrt{k/m}$ (frecuencia natural) y $\zeta = c/(2\sqrt{km})$ (factor de amortiguamiento). Con $\zeta = 1$ se tiene **amortiguamiento crítico**: converge lo más rápido posible sin sobreimpulso (en tiempo continuo). En el controlador se usa $\zeta = 1$ y $\omega_n = 6$ rad/s.

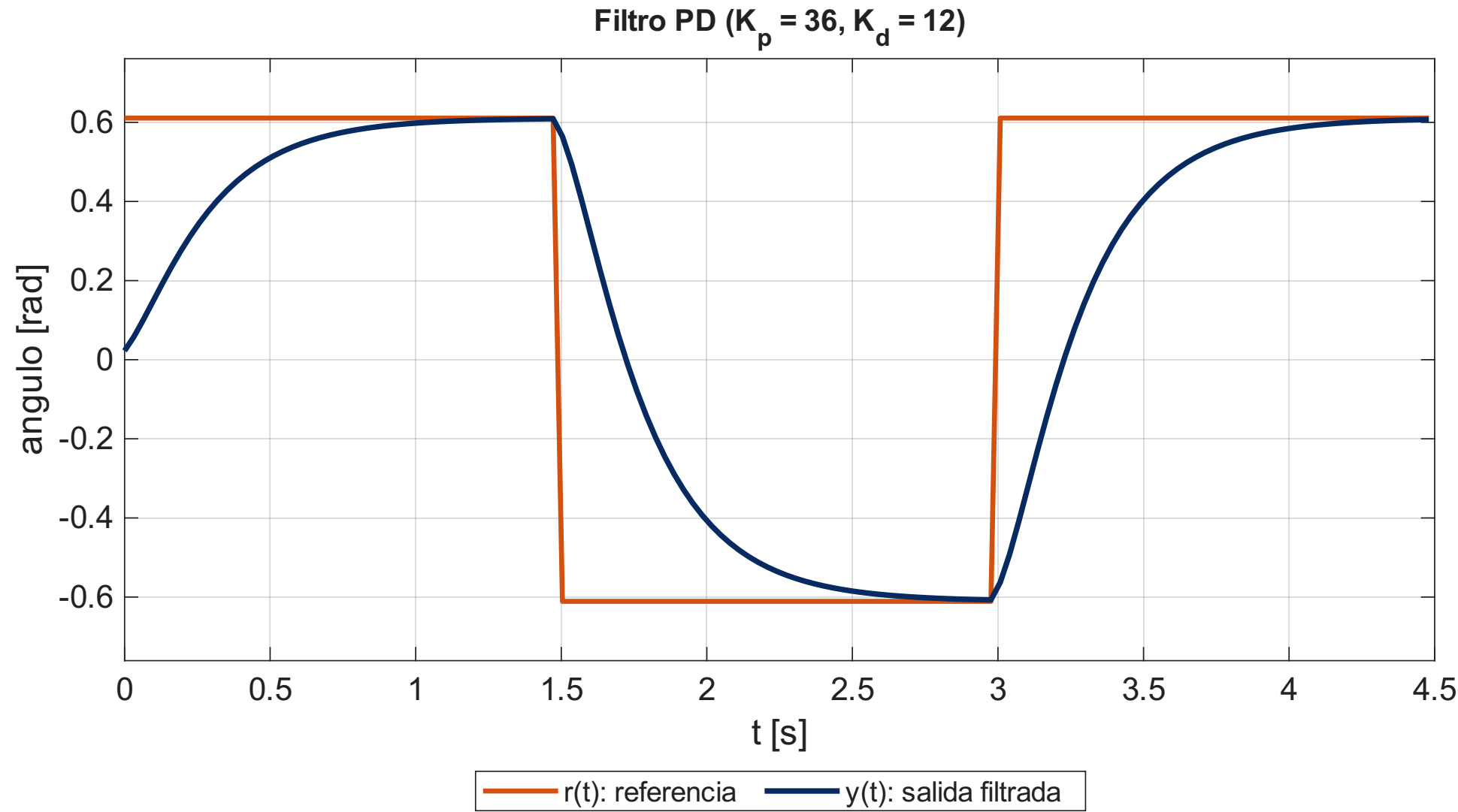
Discretización (Euler): $a[k] = \omega_n^2(r[k] - y[k]) - 2\zeta\omega_n v[k]$; $v[k+1] = v[k] + a[k]\Delta t$; $y[k+1] = y[k] + v[k+1]\Delta t$. Al ser una aproximación discreta, puede aparecer un pequeño sobreimpulso numérico aunque $\zeta = 1$ -por eso también se satura y antes de `setPosition`.



Filtro PD como suavizador

sofia_controller_pd

Espacio reservado – reproducir aquí el video de la simulación en Webots



Filtro PD como suavizador de trayectoria

La misma idea, planteada como una **ley de control** en vez de un modelo físico: una acción proporcional-derivativa que empuja y hacia r .

$$\begin{aligned} u &= K_p(r - y) - K_d v \\ v[k+1] &= v[k] + u[k]\Delta t \\ y[k+1] &= y[k] + v[k+1]\Delta t \end{aligned} \quad (12)$$

donde K_p es la ganancia proporcional, K_d la ganancia derivativa y v la velocidad interna del filtro (un estado auxiliar, no la velocidad real del motor). En el controlador: $K_p = 36$, $K_d = 12$.

Con $K_p = \omega_n^2$ y $K_d = 2\zeta\omega_n$, esta ecuación es **idéntica** a la del resorte-amortiguador crítico -mismo filtro, dos interpretaciones distintas (dinámica de un sistema físico vs. ley de control). Este puente es intencional: conecta el tema de filtros con el de control PD, que se aborda en la siguiente sesión.

Síntesis comparativa

Filtro	Categoría	Parámetros	Idea clave
Media móvil	Señal (FIR)	$N = 15$	Promedio de las últimas N muestras de r
EMA	Señal (IIR, 1er orden)	$\alpha = 0,05$	Paso-bajo exponencial, $\tau \approx \Delta t/\alpha$
Butterworth	Señal (IIR, 2do orden)	$f_c = 0,6 \text{ Hz}$	Biquad; puede resonar y requiere saturación
Mediana	Señal (no lineal)	$N = 9$	Robusta a picos; no suaviza escalones
Rate limiter	Cinemático	$v_{max} = w_{max}/2$	Limita Δy por paso ($\Delta_{max} = v_{max}\Delta t$)
Trapezoidal	Cinemático	$v_{max}, a_{max} = 6$	Acelera / crucero / decelera; aceleración discontinua
Perfil en S	Cinemático	$T = 1 \text{ s}$	Polinomio quintico; velocidad y aceleración suaves
Resorte-amortiguador	2do orden	$\zeta = 1, \omega_n = 6$	ODE crítica, sin sobreimpulso (continuo)
PD como filtro	2do orden	$K_p = 36, K_d = 12$	Idéntico al anterior; puente a control PD



Los 9 filtros son de **lazo abierto**: solo usan $r[k]$ y su propio estado interno. Ninguno lee la posición real del servomotor.